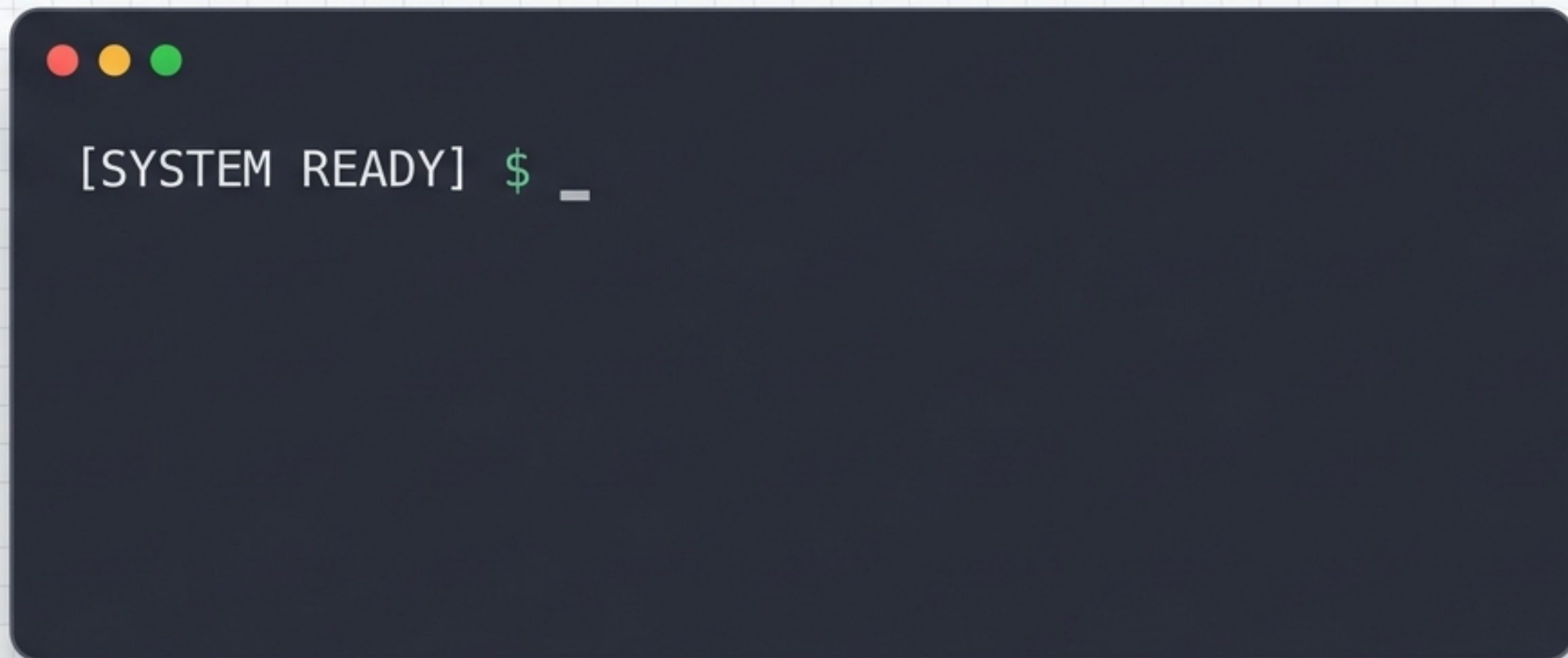


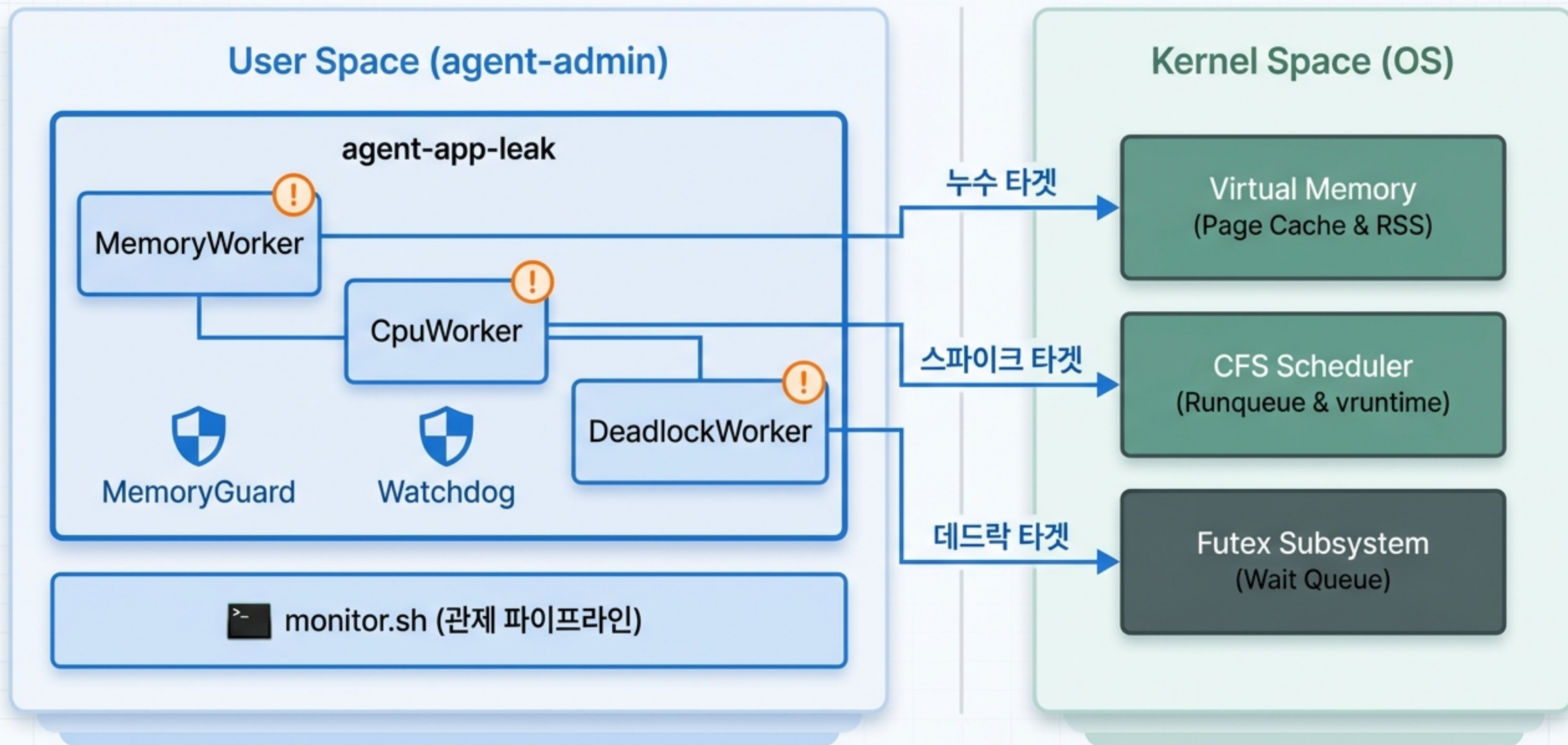
Linux & OS 시스템 리소스 트러블슈팅

심층 기술 평가(Eval) 방어 보고서



Target: agent-app-leak | Status: Analyzed

격리된 컨테이너 환경의 3대 치명적 리소스 장애 시나리오



[Q1] 가상 메모리 노이즈 배제 및 순수 물리 메모리(RSS) 정밀 타겟팅 기법



```
ps -p "$PID" -o rss= 2>/dev/null | awk '{printf "%.0f", $1/1024}'
```



1. 데이터 추출 (ps -o rss=)

헤더를 배제하고 대상 PID의 순수 KB 단위 수치만 추출하여 파싱 오류 원천 차단



2. 단위 환산 (awk \$1/1024)

추출된 KB 값을 운영자가 직관적으로 인지 가능한 MB 단위로 환산



3. 정수 포매팅 (printf "%.0f")

소수점을 제거하여 깔끔한 정수형 로그 레코드 생성

✓ Cross-Check 검증

보조 지표로 `ps -o %mem=` 을 병행하여
시스템 전체 가용량 대비 점유율 교차 검증

[Q2] 관측 목적에 따른 3가지 CPU 진단 도구의 교차 적용



누적 관제 파싱용 (스크립트)

```
ps -p "$PID" -o %cpu=
```

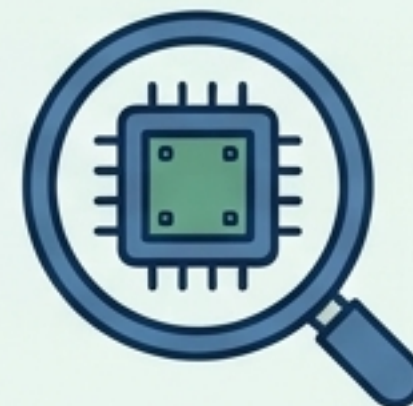
- 프로세스 생명주기 전체의 누적 평균 부하 추출
- monitor.sh 내 자동화 로깅에 최적화된 텍스트 출력



순간 스파이크 캡처용 (실시간)

```
top -p "$PID" -b -n 1
```

- 배치 모드(-b) 및 1회 출력(-n 1) 옵션 적용
- 제어 문자 왜곡 없이 순간적인 실시간 CPU 급증 캡처



단일 코어 독점 진단용 (스레드)

```
ps -p "$PID" -L -o pid,tid,pcpu
```

- -L 옵션으로 경량 프로세스(LWP/스레드) 단위 분해
- 특정 단일 스레드(CpuWorker)의 코어 100% 독점 증명

[Q3] 단편적 판단을 배제한 4단계 논리적 교착상태(Deadlock) 진단 파이프라인

1. 생존 확인
(Crash 배제)

```
ps -ef
```

PID가 정상적으로 조회됨
(비정상 종료 아님 확정)

2. 활성 확인
(Busy-waiting 배제)

```
top -p <PID>
```

CPU 0.0%, 메모리 I/O 변동 0
(무한 연산 대기 아님 확정)

3. 커널 대기 추적
(Futex 고착)

```
ps -L -o wchan
```

스레드 대기 채널이 futex_wait_queue
에 영구 고착 (락 대기 확정)

4. 로그 교차검증
(상호 대기)

```
tail -n 20
```

로그 갱신 중단, 마지막 메시지가 WAITING...
BLOCKED (데드락 최종 확정)

[Q4] 커널 패닉과 연쇄 장애를 방어하는 MemoryGuard의 자가 희생 원리

연쇄 붕괴 시나리오
(Cascading Threat)

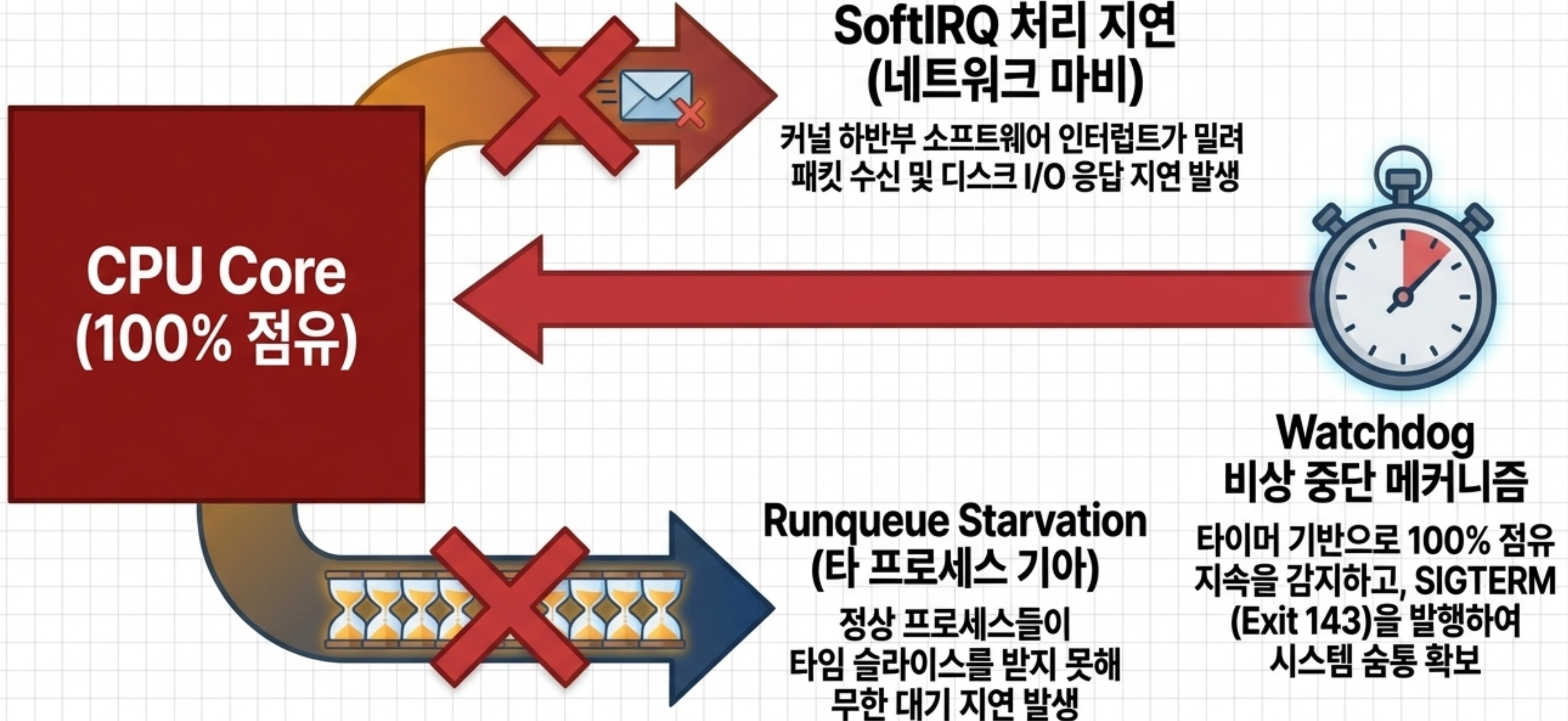


결과: Nginx, DB 등 핵심 인프라
무고한 희생 발생

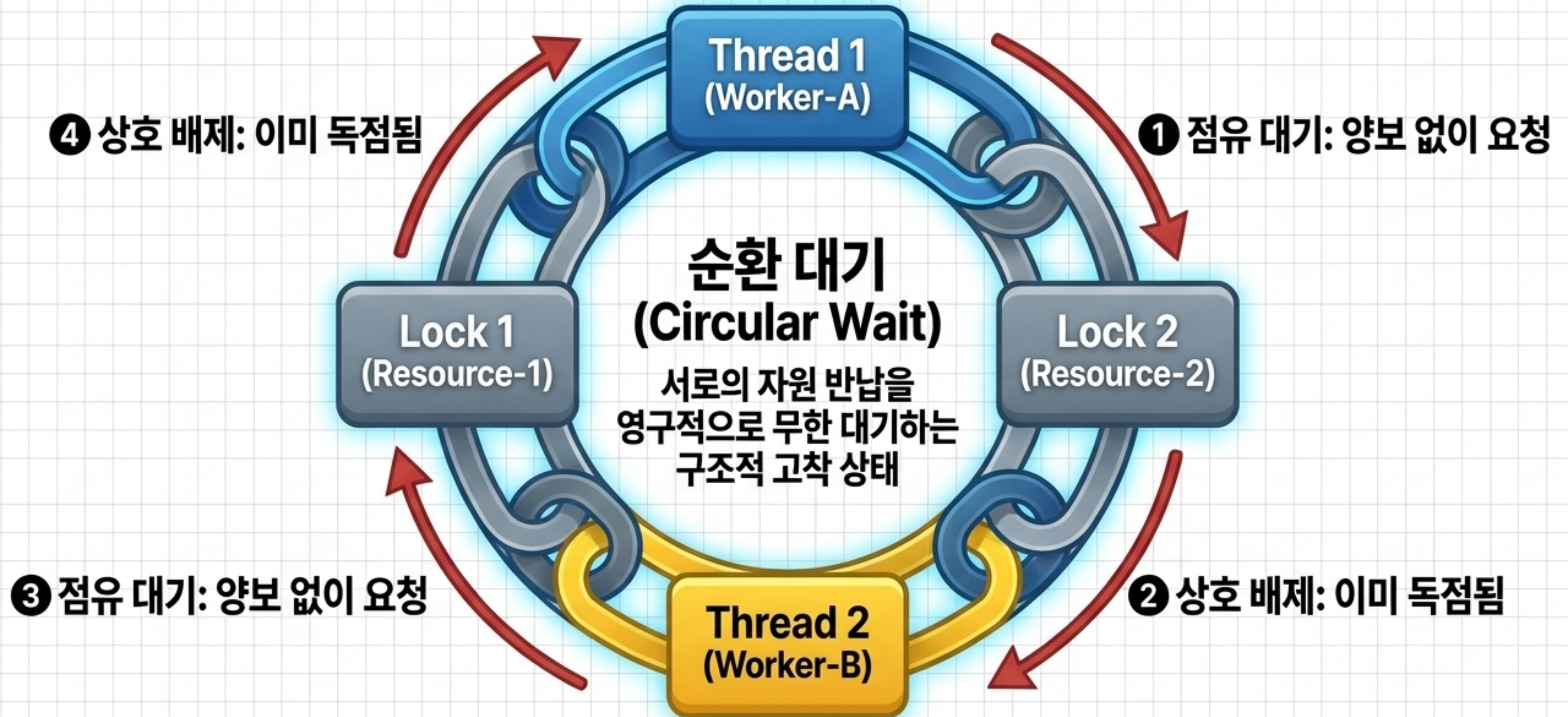
MemoryGuard 선제 개입
(자가 방어)

- 임계치(MEMORY_LIMIT) 도달 선제 감지
- 커널 OOM-Killer 개입 직전, 자신에게 SIGKILL (Exit 137) 발행
- 결과: 해당 프로세스만 Self-Termination 하여 호스트 전체 가용성 완벽 보호

[Q5] 단일 프로세스의 CPU 무한 과점유가 초래하는 시스템 기아(Starvation) 현상



[Q6] Coffman의 4대 조건 성립으로 인한 논리적 교착상태(Deadlock) 원리



[Q7] 타임스탬프 기반 스레드 이벤트 매핑을 통한 순환 의존 관계 역추적

[T1 07:35:00.100] Worker-A : Lock-1 획득 성공 (Holding)
[T2 07:35:00.150] Worker-B : Lock-2 획득 성공 (Holding)
[T3 07:35:00.200] Worker-A : Lock-2 요청 -> [BLOCKED] (Held by TID B) !
[T4 07:35:00.250] Worker-B : Lock-1 요청 -> [BLOCKED] (Held by TID A) !

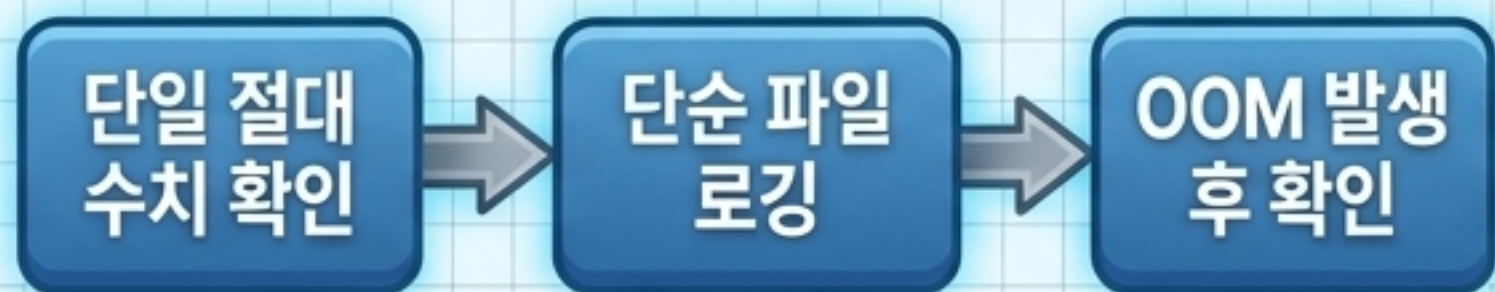
T1 → T2 → T3 → T4 ...

역추적 증명 완료

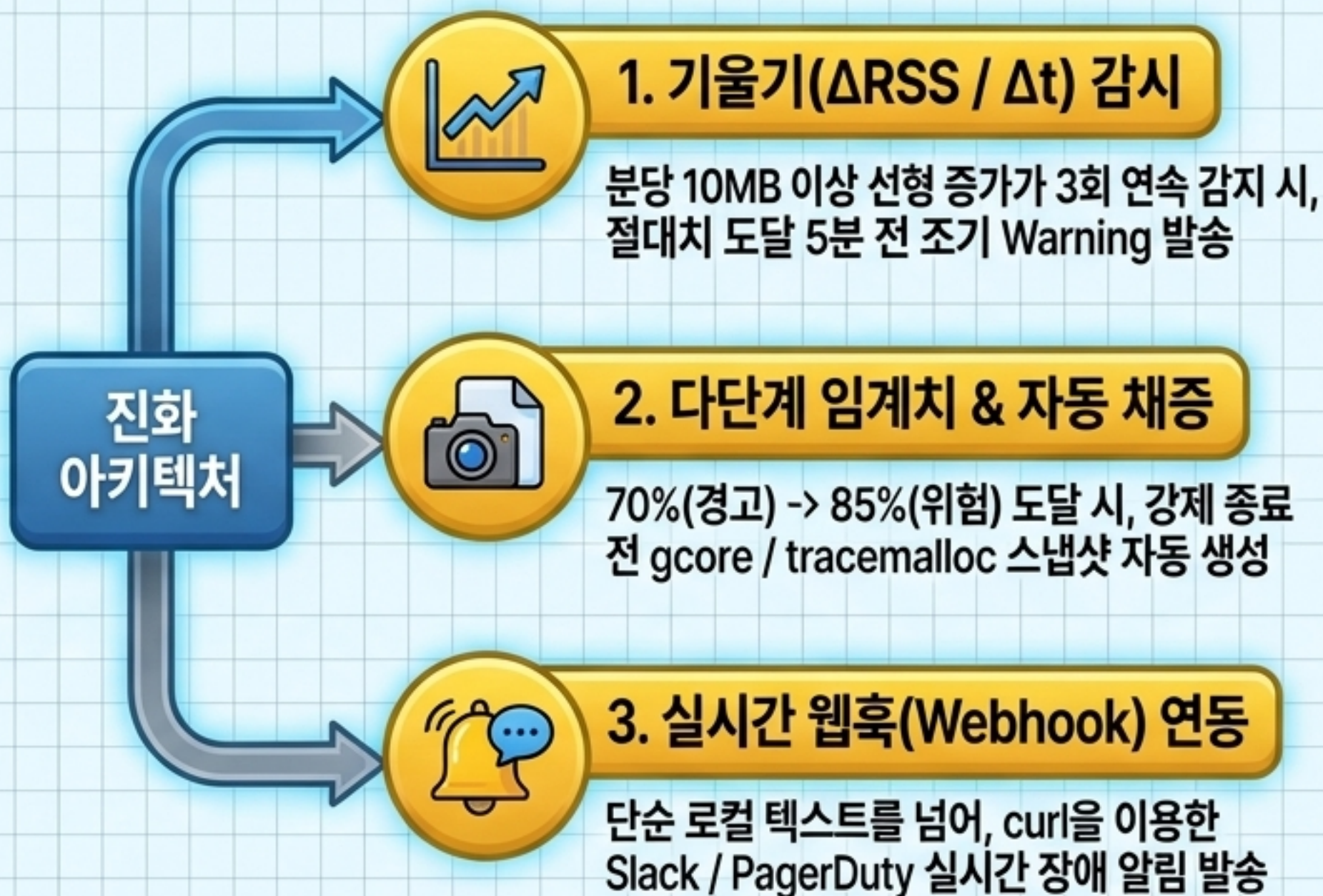
로그 타임스탬프 분석 결과, Worker A -> B 대기와 Worker B -> A 대기가
교차하는 완벽한 크로스 데드락(Cross-Deadlock) 발생 확정

[Q8] 메모리 누수를 장애 발생 전에 조기 탐지하는 예측형 관제 고도화 방안

현재 (단순 사후 기록)



진화 (예측 및 자동화 아키텍처)



[Q9] 관제망을 회피하는 스텔스 장애 'Deadlock'의 치명성과 근본 예방책

OOM / CPU Spike

- 충돌(Crash) 동반
- Liveness Probe 감지 성공
- Pod 자동 재시작으로 일시적 복구 가능



Deadlock (Stealth)

- PID 생존 & CPU 0% 유지
- 헬스체크 관제망 무사 통과 (탐지 불가)
- 요청 누적으로 DB 커넥션 풀 고갈 및
마이크로서비스 연쇄 다운(Cascading Failure) 유발



근본 예방책: 락 순서화 (Lock Ordering Hierarchy)

모든 공유 자원에 고유 ID를 부여하고, 시스템 전역에서 무조건 오름차순(ID가 낮은 순)으로만 락을 획득하도록 소프트웨어 아키텍처 및 코딩 표준을 강제하여 '순환 대기' 원천 차단.

[Q10] OOM vs Deadlock 동시 발생 시의 트리아지(Triage) 행동 강령



1순위: OOM 처리 (응급 지혈 - Infra Survival)

- 🚨 근거: 물리 메모리 고갈은 Swap Thrashing 및 커널 패닉을 유발하여 타 정상 프로세스까지 무차별 킬링하는 급성 파괴 장애.
- 🔍 조치: 즉시 'kill -9'로 프로세스를 강제 종료하여 서버 숨통(가용성) 우선 확보.



2순위: Deadlock 분석 (정밀 진단 - Logical Debugging)

- 근거: 추가적인 리소스 소모 없이 멈춰있는 국소적 무응답 장애이므로 인프라 안정화 이후 접근.
- 조치: 인프라 생존 확보 후, 데드락 프로세스의 스레드 덤프(pstack, gdb)를 채증하여 코드 레벨 원인을 천천히 디버깅.

[Q11] 장애 메커니즘을 근본적으로 타겟팅하는 소스 코드 레벨 리팩토링

Memory Leak

현행 결함
무한 전역 리스트 적재

[LRU Cache & weakref]

개선 아키텍처
최대 크기가 제한된 캐시
자료구조 도입 및 불필요
데이터의 명시적 파기(del).

CPU Spike

현행 결함
동기식 무한 Polling 연산 루프

[Event-Driven]

개선 아키텍처
asyncio.sleep(0) 삽입으로
OS에 타임 슬라이스 강제 양보
및 비동기 이벤트 기반
아키텍처 도입.

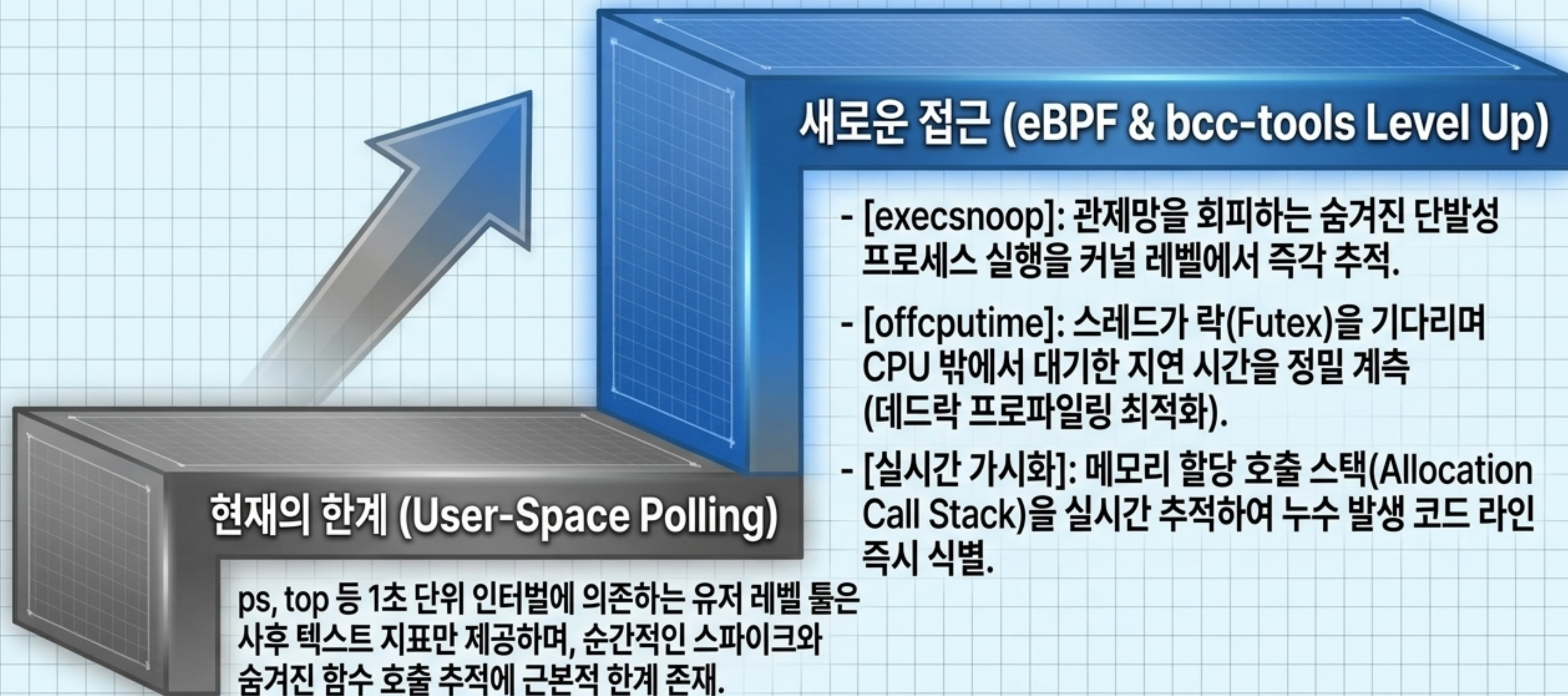
Deadlock

현행 결함
무한 대기 acquire() 락킹

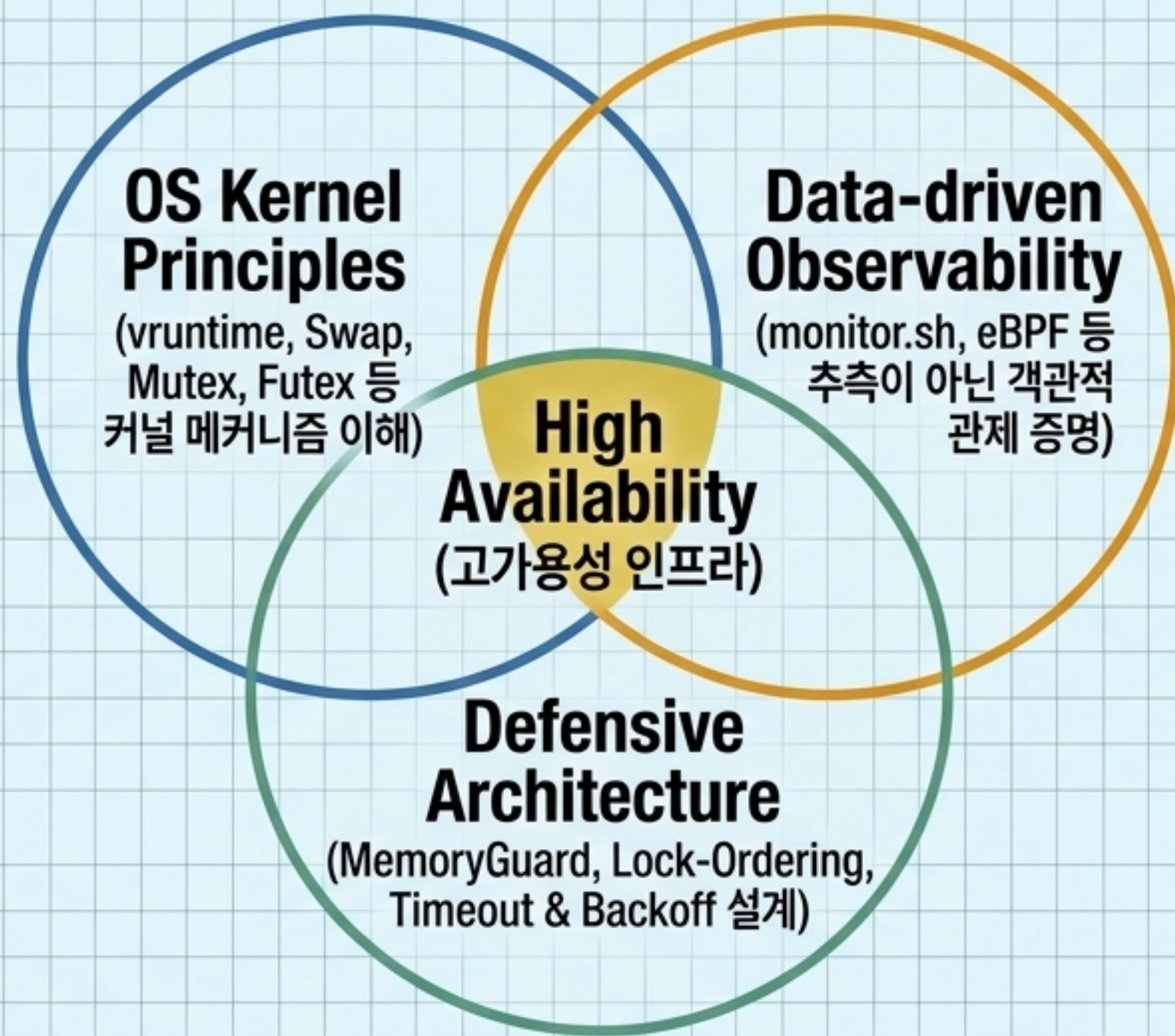
[Timeout & Backoff]

개선 아키텍처
acquire(timeout=N) 적용.
획득 실패 시 기 보유 락 반납 후
지수 백오프(Exponential
Backoff) 재시도 로직 구현.

[Q12] 사후 관찰의 한계를 극복하는 커널 레벨 동적 프로파일링(eBPF) 도입



[Q13] 시스템 리소스 엔지니어링의 3대 본질과 고가용성의 완성



맹목적인 재부팅이 아닌, 관제 데이터 기반의 논리적 원인 규명과 방어적 설계가 결합될 때 비로소 반복되는 장애 사슬을 끊고 진정한 시스템 고가용성을 달성할 수 있습니다.